Five Project Proposals for VISxAI 2026 — Visualization for AI Explainability

A Formal Research Proposal Document for a Solo Author (Web-Based Interactive Explainables)

Executive Summary

This document proposes five interactive "explainables" for the 9th Workshop on Visualization for AI Explainability (VISxAI), co-located with IEEE VIS 2026 in Boston, MA (workshop day in October/November 2026). Two topics are mandated by the brief — **Vision-Language Model (VLM) Interpretability** and **Inference-Time Scaling** — and three additional slots are filled after a gap analysis against the VISxAI Hall of Fame and adjacent landmark explainers (Distill.pub, Google PAIR AI Explorables, the Georgia Tech Polo Club suite). The two suggested topics (Tesla Autopilot and Quantization) were evaluated explicitly: **Quantization is recommended for inclusion** because it is genuinely novel for VISxAI, precomputable, and behavior-focused; **Tesla Autopilot is recommended against** because it is a product/systems story rather than an AI-method explainable, its real models are inaccessible, and it is hard to run credibly in-browser. The remaining two slots go to **Mixture-of-Experts (MoE) Routing** and **Sparse Autoencoders / Feature Steering**, both of which are timely, un-served by existing VISxAI entries, and feasible for one person.

The five proposals, in recommended priority order:

#	Title	Required?	Novelty	Solo Feasibility	Audience Breadth	Wow-Factor
1	**Inside the Machine's Eye** (VLM interpretability)	✅ Required	High	Medium	High	Very High
2	**The Thinking Budget** (inference-time scaling)	✅ Required	Very High	High	High	High
3	**Lossy Minds** (quantization)	Suggested → included	Very High	High	Medium	High
4	**Who Handles This Token?** (mixture-of-experts)	Gap-filling	High	Medium-High	Medium	High
5	**The Microscope** (sparse autoencoders / steering)	Gap-filling	Medium-High	Medium	Medium	Very High

Recommended stack (primary): SvelteKit + D3.js for the interactive layer, `scrollama`/a Svelte `<Scrolly>` component for scrollytelling, Motion One (or Svelte transitions) for animation, Three.js/WebGL only where 3D is essential, and **precomputed JSON traces** as the default data strategy — with **Transformers.js v3 + ONNX Runtime Web (WebGPU)** for live in-browser inference where it adds pedagogical value. This mirrors the exact stack of the 2025 Best-Submission *Transformer Explainer* by Cho, Kim, Karpekov, Helbling, Wang, Lee, Hoover & Chau (Georgia Tech), which runs the GPT-2 small model (124M parameters, converted from Karpathy's nanoGPT to ONNX Runtime for in-browser execution) using "Svelte as a front-end framework and D3.js." The whole site is a static build, trivially dockerizable and Vercel-hostable (a prior VISxAI Hall-of-Fame entry, the K-Means explorable, is itself hosted at `k-means-explorable.vercel.app`).

Part I — Understanding the VISxAI Venue

What VISxAI is and what it rewards

VISxAI solicits "explainables"/"explorables": visual, interactive narratives that teach how an AI technique works. The organizers state the narrative "has to be visual and interactive, and walk readers through a keen understanding on the ML technique or application," and explicitly suggest Distill-style blog posts, Idyll markup, or Jupyter/Observable notebooks. Crucially, **the explainable must stand on its own** — reviewers evaluate the explainable and may choose to ignore any accompanying paper. Each year awards **Best Submissions** and **Honorable Mentions**; historically the best works were invited to extend to Distill.pub. Visxai

The 2026 (9th) edition is organized by Alex Bäuerle (Google DeepMind), Angie Boggust (MIT), Catherine Yeh (Harvard), and Fred Hohman (Apple), with a steering committee of Adam Perer (CMU), Hendrik Strobelt (MIT-IBM Watson AI Lab), and Mennatallah

El-Assady (ETH AI Center). visxai

Key dates (VISxAI 2026, from visxai.io)

Milestone	Date
Call for Participation	May 5, 2026
Submission Deadline (anywhere on Earth)	August 3, 2026
Reviews Submitted	September 8, 2026
Author Notification	September 14, 2026
Camera-Ready Deadline	October 16, 2026
Workshop	IEEE VIS 2026, Boston, MA (October/November)

Submission logistics

Submissions go through the **PCS system** (precisionconference.com): Society **VGTC** → Conference **VIS 2026** → Track **"VIS 2026 VISxAI Workshop Blog Posts."** You provide title, all authors, an abstract, and **either a link to the live explainable or an uploaded document** (both allowed). This means a solo author should treat the deployed URL (a Vercel deployment) as the primary artifact. visxai visxai

Gap analysis — the VISxAI Hall of Fame and adjacent landmarks

What has already been done (saturated):

- **Transformers / LLM text generation:** *Transformer Explainer* (Cho et al., VISxAI 2025 Best Submission; runs GPT-2 small live in-browser via ONNX). Heavily saturated.
- **Diffusion / text-to-image:** *Diffusion Explainer* (Lee et al., Polo Club) and *Diffusion Explorer* (Helbling).
- **Language-model probing:** *What Have Language Models Learned?* (Adam Pearce, PAIR, VISxAI 2021); *Do Machine Learning Models Memorize or Generalize?* (grokking; PAIR, VISxAI 2023). visxai
- **Mechanistic interpretability of LLMs:** *Can Large Language Models Explain Their Internal Mechanisms?* (Patchscopes; PAIR, VISxAI 2024). visxai
- **Foundational ML concepts:** dimensionality reduction (Conlen & Hohman 2018), Gaussian processes (Görtler et al. 2018), K-means (Ang 2022), PAC learning (Cashman 2023), CNNs (CNN Explainer), GANs (GAN Lab). visxai
- **Multi-modal comparison:** *Understanding and Comparing Multi-Modal Models* (Humer et al., VISxAI 2023) — CLIP-style embedding comparison, not internal attention/logit-lens. Visxai
- **AlphaFold:** *The Illustrated AlphaFold* (Simon & Silberg, VISxAI 2024). Visxai

Under-explored gaps (opportunities):

1. **VLM internals** — how a model fuses pixels and words. VL-InterpreT (Intel, CVPR 2022 best demo) and LVLM-Interpret (CVPR 2024 workshop) exist as *research/analyst tools*, but there is **no pedagogical scrollytelling explainable** of VLM cross-modal attention and logit-lens for a general audience.
2. **Inference-time scaling / reasoning models** — the single hottest 2024-2026 AI topic (o1, DeepSeek-R1). Research/analyst tools exist (iToT for Tree-of-Thoughts; Hippo for interactive reasoning, UIST 2025) but **no VISxAI-style explainable** of *why* spending compute at test time works, scaling curves, and the search/verification machinery.
3. **Quantization** — a behavior-and-numerics topic with **zero prior VISxAI coverage**.
4. **Mixture-of-Experts routing** — powers Mixtral/DeepSeek/GPT-class models; **no interactive explainable of token→expert routing** at VISxAI.
5. **Sparse autoencoders / feature steering** — Anthropic's "Golden Gate Claude" made this famous; **no explorable** lets a general audience *steer* a model by turning features up/down.

Part II — Recommended Technology Stack

Primary stack and justification

Framework: SvelteKit. Svelte compiles away the framework, produces tiny bundles, and has fine-grained reactivity ideal for tying sliders/scroll position to live visual updates. It is the framework behind *Transformer Explainer* (Svelte + D3), *CNN Explainer*, and other award-winning explainables, so it is a proven fit for the venue's aesthetic. SvelteKit's static adapter yields a fully static site.

Visualization: D3.js for bespoke SVG encodings (attention heatmaps, Sankey flows, sampling trees, probability streams, weight-distribution histograms). D3 pairs cleanly with Svelte (Svelte owns the DOM; D3 owns scales/layouts/paths).

Scrollytelling: `scrollama` (IntersectionObserver-based) or a lightweight Svelte `<Scrolly>` component (e.g., the MIT Interactive Visualization & Society lab pattern / Pudding-style starter). Sticky-graphic + scrolling-prose is the canonical VISxAI narrative device.

Animation: Svelte's built-in transitions/tweened/spring stores for most motion; **Motion One** (Web Animations API) for richer choreography; reserve GSAP only if needed. This delivers the "custom rich animations" the brief requires.

3D / WebGL: Three.js only where 3D genuinely helps — e.g., a rotating occupancy-style voxel grid or a 3D embedding-space fly-through. Otherwise avoid the complexity budget.

In-browser inference (where used): Transformers.js v3 (`@huggingface/transformers`) over **ONNX Runtime Web** with the **WebGPU** backend (`device: 'webgpu'`), falling back to WASM. Enable WebGPU via Chrome/Edge 113+ (FP16 since Chrome 121); set COOP/COEP headers for multi-threaded WASM. Use quantized ONNX weights (q4/q8) to keep downloads to a few hundred MB.

Critical architectural decision: precomputed traces vs. live inference

A dedicated feasibility investigation surfaced a decisive constraint: **the pre-exported ONNX graphs used by Transformers.js only emit logits and the KV-cache — not per-layer attention matrices or hidden states**, and enabling `output_attentions`/`output_hidden_states` at ONNX export time is documented as buggy (HuggingFace `transformers` issue #16282, which notes the export "results for hidden states/attentions are incorrect if enabled" due to the tuple-returning nature of the forward functions). Therefore:

> **Default data strategy = precompute internals offline, ship as static JSON.** Run the target model once in Python (PyTorch + HuggingFace, `output_attentions=True, output_hidden_states=True, return_dict_in_generate=True`), compute attention-rollout / logit-lens / routing / weight-distribution traces for a curated set of examples, serialize to JSON, and drive the visualizations from that. This is exactly how VL-InterpreT and LVLM-Interpret operate (extract to a store, then visualize). One handling caveat: for generation step 0 the attention tensor is the full prompt self-attention `(1, heads, seq_len, seq_len)`; from step 1 onward it is `(1, heads, 1, ctx_len)` — build the trace format to accommodate the first-token shape.

> **Live inference = optional interactive garnish.** Where live generation adds pedagogical payoff (typing a new prompt, watching tokens stream, moving a temperature slider), run a *small* model live via Transformers.js/WebGPU — decoupled from the internals view. Per the SmolVLM technical report (arXiv:2504.05299), the authors "demonstrate the ability to efficiently run these models locally within a browser environment via WebGPU, with the 256M variant achieving up to 80 decode tokens per second on a 14-inch MacBook Pro (M4 Max)," using under 1 GB of VRAM; official Hugging Face WebGPU demos exist (e.g., `webml-community/smolvlm-realtime-webgpu`, `HuggingFaceTB/SmolVLM-256M-Instruct-WebGPU`, and `Xenova/experimental-moondream-webgpu`). GPT-2 small already runs live in *Transformer Explainer*.

Deployment (dockerizable + Vercel)

`npm run build` with `@sveltejs/adapter-static` (or `adapter-vercel`) produces static assets. A two-stage Dockerfile (Node build → static file server / nginx) satisfies the dockerizable requirement; the same build deploys to Vercel with zero server code. Large model/JSON assets are served from `static/` or a CDN; set cross-origin isolation headers (COOP/COEP) in `vercel.json` for WebGPU + threaded WASM. A prior VISxAI Hall-of-Fame entry (the K-Means explorable) is itself Vercel-hosted, confirming venue fit.

Per-project stack variations

- **VLM, MoE, SAE, Quantization:** primarily precomputed JSON + D3/Svelte; SAE and VLM optionally add a live SmolVLM-256M WebGPU demo.

- **Inference-Time Scaling:** entirely precomputed traces + D3 tree/stream visualizations; no live model required (avoids API keys and cost). Optional live small-model best-of-N via Transformers.js as a stretch.

Part III — The Five Proposals

Proposal 1 — *Inside the Machine's Eye: How AI Reads a Picture and a Sentence at Once*

(Required topic: Vision-Language Model interpretability)

AI concept taught. How a vision-language model (VLM) converts an image into tokens, fuses them with text tokens through the transformer stack, and arrives at an answer. Learning objectives: (1) images become patch tokens alongside word tokens; (2) **cross-modal attention** lets text tokens "look at" image regions (and vice versa); (3) the **logit lens** shows visual-token representations gradually becoming language-like across layers; (4) attention is a *plausible but not always faithful* explanation — a deliberately taught caveat.

Need / gap analysis. VISxAI has covered text transformers exhaustively (*Transformer Explainer*, 2025) and multi-modal *embedding comparison* (Humer et al., 2023, CLIP-style), but **no explainable teaches what happens inside a VLM when it answers a question about an image.** Research tools exist — **VL-InterpreT** (Intel Labs; CVPR 2022 Best Demo; language↔vision attention heatmaps: pick an image patch to see contributing text tokens, or a text token to see an attention heatmap over the image) and **LVLM-Interpret** (Intel, CVPR 2024 workshop; raw attention, relevancy maps, and causal graphs for LLaVA) — but they are analyst dashboards, not narrated pedagogy for a broad audience. Recent findings give this explainable real substance: Neo, Ong, Torr, Geva, Krueger & Barez, "Towards Interpreting Visual Information Processing in Vision-Language Models" (ICLR 2025; arXiv:2410.07149), report that "visual token representations become increasingly interpretable in the vocabulary space across layers" and that "object identification accuracy drops by over 70% when object-specific tokens are removed" — i.e., object information is localized in the visual tokens at the object's spatial position. This proposal turns those research insights into a guided, interactive story — the novelty is **pedagogy + narrative + a general-audience "steering the gaze" interaction**, not a new research tool.

Target audience. ML students and practitioners new to multimodality; technically curious general readers; educators.

Interactive visualization design.

- **Hero view — "the gaze overlay":** a photo beside a question ("What color is the umbrella?"). As the model answers token-by-token, the image is overlaid with a **cross-modal attention heatmap** (24×24 patch grid) showing which regions the current output token attends to. A scrubber steps through generated tokens; the heatmap animates.
- **Layer slider — "the logit lens":** for a chosen visual token, project its hidden state at each layer through the unembedding matrix and show a **streamgraph of the top vocabulary words** it "means" at each depth — visualizing pixels turning into words.
- **Head explorer:** small-multiples grid of attention heads; click a head to see whether it does language→vision, vision→language, or within-modality attention (the four VL-InterpreT quadrants), reframed for learners.
- **Faithfulness challenge:** an interaction where occluding the "attended" region sometimes *doesn't* change the answer — teaching that attention ≠ explanation.

Narrative / pedagogy (scrollytelling arc).

1. *A question about a picture* — set up the task.
2. *Everything becomes tokens* — patches + words in one sequence.
3. *Looking across modalities* — cross-modal attention, gaze overlay.
4. *From pixels to words* — logit lens streamgraph.
5. *Not all heads are equal* — head roles.
6. *Does attention tell the truth?* — faithfulness caveat (misconception addressed: "the heatmap is the reason").
7. *Try it yourself* — optional live SmolVLM demo.

Methodology. Precompute traces offline with **LLaVA-1.5-7B** (the de-facto standard for VLM interpretability — richest set of copyable recipes; clean 24×24 patch grid), using `output_attentions`/`output_hidden_states` and a logit-lens matmul (`logits_l = W_U · h_l`) on visual-token hidden states; serialize per-example JSON (attention tensors, rollout, logit-lens top-k per layer).

Curate ~8–12 image/question pairs. Optional live layer: **SmolVLM-256M** via Transformers.js v3 + WebGPU (under 1 GB VRAM; up to ~80 tok/s on M4 Max) for free-form Q&A, decoupled from the internals view. D3 for heatmaps/streamgraphs; Svelte for state.

Feasibility (solo). *Medium.* Main risk is attention-tensor payload size (VLMs have many visual tokens → large JSON); mitigate by downsampling heads/layers shown, quantizing JSON, and limiting curated examples. Offline extraction is well-trodden. **MVP:** gaze overlay + logit lens on precomputed examples. **Stretch:** live SmolVLM demo, head explorer, faithfulness challenge. Estimated effort: 6–9 focused weeks.

Novelty statement. First VISxAI explainable to *narrate* VLM cross-modal attention and the visual-token logit lens for a general audience, translating VL-InterpreT / LVLM-Interpret / Neo-et-al. research into guided pedagogy with a "steer the gaze" interaction and an explicit faithfulness lesson.

Proposal 2 — *The Thinking Budget: Why Letting AI Think Longer Makes It Smarter (Until It Doesn't)*

(Required topic: Inference-time scaling / test-time compute)

AI concept taught. How and why allocating more compute at *inference* time — rather than training a bigger model — improves reasoning. Learning objectives: (1) **chain-of-thought (CoT)** = sequential scaling (think longer); (2) **best-of-N / self-consistency** = parallel scaling (think wider, then vote/verify); (3) **process reward models & tree search** guide which thoughts to expand; (4) **compute-optimal allocation** depends on problem difficulty; (5) more thinking is **not monotonically better** — over-thinking can hurt.

Need / gap analysis. This is arguably the defining AI theme of 2024–2026 (OpenAI o1/o3, DeepSeek-R1), yet **no VISxAI explainable exists** for it. Existing interactive tools are research prototypes for *practitioners* — iToT (Tree-of-Thoughts explorer) and Hippo (Interactive Reasoning, UIST 2025) — not narrated explainers of the underlying scaling laws for a general audience. The proposal is grounded in landmark results: Snell, Lee, Xu & Kumar, "Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters" (arXiv:2408.03314), show that a compute-optimal strategy "can improve the efficiency of test-time compute scaling by more than 4x compared to a best-of-N baseline," and that "on problems where a smaller base model attains somewhat non-trivial success rates, test-time compute can be used to outperform a 14x larger model." DeepSeek-R1 improved AIME accuracy dramatically via extended CoT (reported as rising to ~71%, and ~86.7% with majority voting).

Target audience. ML practitioners, students, and technically literate policymakers/decision-makers who keep hearing "reasoning models" and "test-time compute" without a mental model.

Interactive visualization design.

- **The compute-budget slider:** a single slider for "thinking budget" (tokens / N samples / search width). As the user drags it, an **accuracy-vs-compute curve** fills in, and a live example problem's solution updates.
- **Sequential vs. parallel toggle:** left = one long CoT chain growing token-by-token (a "token probability stream"); right = **best-of-N sampling tree** with a verifier scoring branches, the winner highlighted by majority vote / reward.
- **Tree-search explorer:** an interactive MCTS/beam-style tree where nodes are partial reasoning steps colored by a process-reward-model score; the user expands/prunes and watches the answer change.
- **Difficulty dial:** switch between easy/medium/hard prompts to show that the *compute-optimal* strategy shifts (easy → brief self-revision; hard → wide search) — the core Snell et al. insight.
- **The over-thinking cliff:** push the budget past the sweet spot and watch accuracy *drop*, teaching that longer ≠ better.

Narrative / pedagogy (scrollytelling arc).

1. *Two ways to get smarter* — train bigger vs. think longer.
2. *Thinking longer* — CoT as sequential compute (token stream).
3. *Thinking wider* — best-of-N, self-consistency voting.
4. *A judge for thoughts* — process reward models, verifiers.
5. *Searching the tree of thoughts* — MCTS/beam interaction.
6. *Spend compute wisely* — compute-optimal vs. difficulty (Snell curves).
7. *David beats Goliath* — small model + compute > 14× larger model.
8. *When more is worse* — the over-thinking cliff; faithfulness of CoT caveat.

Methodology. Fully precomputed — no live LLM (avoids API keys, cost, latency). Generate reasoning traces offline for a handful of math/logic problems using an open reasoning model (e.g., a DeepSeek-R1-distilled model) at varying budgets and N; record per-branch verifier scores, majority votes, accuracy-vs-compute points. Serialize to JSON. D3 for trees (d3-hierarchy), streams, and curves; Svelte for slider/toggle state; Motion One for branch-growth animation. Optional stretch: live best-of-N with a tiny model via Transformers.js.

Feasibility (solo). *High.* No model runs in the browser by default; all heavy lifting is offline and one-time. Chief risk is scope creep across too many methods — mitigate by anchoring on CoT, best-of-N, and one tree-search view. **MVP:** budget slider + accuracy curve + sequential-vs-parallel toggle on precomputed traces. **Stretch:** tree-search explorer, over-thinking cliff, live small-model demo. Estimated effort: 5–7 weeks.

Novelty statement. First VISxAI explainable to teach test-time compute scaling laws — CoT, best-of-N, verifier-guided search, and compute-optimal allocation — as an interactive, narrated story, making the Snell-et-al. and o1/R1 results tangible via a single "thinking budget" the reader controls.

Proposal 3 — *Lossy Minds: What a Model Forgets When You Shrink It*

(Suggested topic — recommended for inclusion: Quantization)

AI concept taught. How quantization compresses model weights (FP16 → INT8 → INT4 and below), why it works, and how it changes behavior. Learning objectives: (1) weights are numbers with precision; fewer bits = less memory; (2) naive rounding (RTN) loses information non-uniformly; (3) **GPTQ** (layer-wise, second-order) and **AWQ** (activation-aware, protects salient weight channels) recover most quality; (4) there is a **cliff** — INT4 is often near-lossless, but 2-bit can collapse; (5) reasoning models are more fragile under compression. (Medium)

Need / gap analysis. Zero prior VISxAI coverage, and quantization is a perfect visualization subject because it is fundamentally about *numbers and distributions*. It is also highly relevant: a 70B model needs ~140 GB in FP16 but ~35–40 GB in INT4, which is exactly what makes local/browser inference possible. Concrete, visualizable results anchor the story: Lin et al., "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration" (MLSys 2024; arXiv:2306.00978), show that "weights are not equally important: protecting only 1% of salient weights can greatly reduce quantization error" and that AWQ works by searching "for the optimal per-channel scaling that protects the salient weights by observing the activation, not weights" (the paper cites a 0.1%–1% salient range). AWQ's INT4-g128 keeps accuracy nearly intact while RTN/GPTQ degrade more, and 2-bit quantization collapses GSM8K accuracy to near zero (Evaluating Quantized LLMs, arXiv:2402.18158). (VRLA Tech)

Target audience. Practitioners deploying models on constrained hardware; students; the growing "local LLM" hobbyist community.

Interactive visualization design.

- **The bit-depth slider:** drag FP16 → INT8 → INT4 → INT3 → INT2 and watch a **weight-distribution histogram** snap onto a coarsening grid of quantization levels in real time, with the quantization error shaded.
- **Salient-channel spotlight (AWQ):** highlight the ~1% of channels with large activations; toggle "protect salient weights" on/off and see reconstruction error and downstream accuracy change — the AWQ intuition made visual.
- **Method bake-off:** side-by-side RTN vs. GPTQ vs. AWQ on the same layer, showing error and a downstream metric (e.g., GSM8K / captioning score) — including the dramatic **2-bit collapse**.
- **Behavior diff:** before/after generations on the same prompt at each bit-width, so readers *see* where outputs start to drift or break.
- **Memory meter:** a live "fits on your GPU?" gauge (140 GB → 35 GB as bits drop).

Narrative / pedagogy (scrollytelling arc).

1. *Models are just big piles of numbers.*
2. *Precision costs memory* — the memory meter.
3. *Rounding, naively* — RTN and where it hurts.
4. *Not all weights matter equally* — salient channels (AWQ).
5. *Smarter compression* — GPTQ vs. AWQ bake-off.
6. *The cliff* — INT4 fine, INT2 collapses.

7. *Fragile reasoners* — why reasoning/VLM models degrade more.
8. *Why you can run AI in your browser* — tie back to on-device inference.

Methodology. Fully precomputed. Offline in Python, quantize a small open model (e.g., a 1–8B model) with RTN, GPTQ, and AWQ at multiple bit-widths; export weight histograms, per-method reconstruction error, and downstream metrics to JSON. The histogram-coarsening and salient-channel interactions can be computed *live in JS* on small sampled weight arrays (pure math, no model). D3 for histograms/bar charts; Svelte for sliders/toggles.

Feasibility (solo). *High.* Almost no in-browser ML; the live interactions are lightweight numeric transforms on precomputed arrays. Risk: sourcing clean before/after generations — mitigate by curating a fixed prompt set offline. **MVP:** bit-depth slider + histogram + memory meter + method bake-off. **Stretch:** salient-channel spotlight, behavior diff, reasoning-model fragility panel. Estimated effort: 4–6 weeks (lowest-risk of the five).

Novelty statement. First interactive explainable of quantization for VISxAI, turning GPTQ/AWQ and the precision-vs-behavior trade-off into a hands-on "shrink the model and watch it think" experience — a genuinely fresh topic for the venue.

Proposal 4 — *Who Handles This Token? Inside a Mixture-of-Experts Model*

(Gap-filling slot)

AI concept taught. How sparse Mixture-of-Experts (MoE) models route each token to a small subset of expert sub-networks, achieving large capacity at low active compute. Learning objectives: (1) the FFN layer is replaced by many experts + a router/gate; (2) **top-K routing** sends each token to its highest-scoring experts; (3) only a fraction of parameters are *active* per token; (4) experts specialize by token/context, not tidy human "subjects"; (5) load balancing keeps experts evenly used.

Need / gap analysis. MoE underpins many frontier models (Mixtral, DeepSeek-V3/R1, and others), yet **no VISxAI explainable covers routing.** Existing artifacts are static visual guides (Maarten Grootendorst's illustrated MoE) or research tools (MixtureKit's Streamlit routing visualizer) — none is a narrated, interactive scrollytelling explainable. The routing mechanism is inherently visual (token→expert assignments, heatmaps of gate weights), making it ideal. The canonical concrete example, from Jiang et al., "Mixtral of Experts" (arXiv:2401.04088): Mixtral 8×7B has 46.7B total parameters but 12.9B active per token — verbatim, "each token has access to 47B parameters, but only uses 13B active parameters during inference" (top-2 of 8 experts per layer), and it "outperforms or matches Llama 2 70B and GPT-3.5 across all evaluated benchmarks."

Target audience. Students and practitioners who read "8×7B" or "active parameters" and want an intuition; ML-curious general readers.

Interactive visualization design.

- **Routing flow:** type a sentence; each token animates along a **Sankey-style flow** to its top-K experts at a chosen layer, colored by gate weight.
- **Gate-weight heatmap:** tokens (rows) × experts (columns), cells shaded by routing probability, with the top-K selected cells outlined.
- **Layer stepper:** step through layers to see how a token's chosen experts change with depth.
- **Specialization explorer:** aggregate routing over many tokens to reveal what each expert tends to handle (e.g., punctuation, numbers, code) — dispelling the "expert = a topic like biology" misconception.
- **Capacity/compute meter:** a dial contrasting total vs. active parameters (e.g., 47B total / 13B active), showing why MoE is efficient.

Narrative / pedagogy (scrollytelling arc).

1. *One big brain, or many small specialists?*
2. *The router decides* — gating scores.
3. *Top-K: pick a few, ignore the rest* — routing flow.
4. *Active vs. total parameters* — the efficiency trick (Mixtral 47B/13B).
5. *Depth changes the routing* — layer stepper.
6. *What do experts actually specialize in?* — specialization explorer + misconception.
7. *Keeping experts busy* — load balancing.

Methodology. Precomputed. Offline, run an open MoE (e.g., a small Mixtral/OLMoE/Qwen-MoE variant) on curated sentences with router logits captured per layer; export top-K assignments and gate weights to JSON. Aggregate specialization stats offline. All interactions then run on static JSON. D3 for Sankey/heatmap; Svelte for input/steppers; Motion One for token-routing animation. (For a limited live demo, a tiny MoE could run via Transformers.js, but precomputed is the safe default.)

Feasibility (solo). *Medium-High.* Offline router-logit extraction is straightforward with HuggingFace hooks. Risk: making routing legible at scale (many tokens × many experts) — mitigate with short sentences, one layer at a time, and aggregation views. **MVP:** routing flow + gate heatmap for precomputed sentences + capacity meter. **Stretch:** layer stepper, specialization explorer, free-text input via live tiny MoE. Estimated effort: 5–7 weeks.

Novelty statement. First interactive MoE routing explainable at VISxAI — a "follow your token to its expert" experience that makes sparse activation and expert specialization tangible, correcting the common misconception that experts map to human subject areas.

Proposal 5 — *The Microscope: Finding — and Turning — the Knobs Inside a Neural Network*

(Gap-filling slot)

AI concept taught. How sparse autoencoders (SAEs) decompose a model's dense, polysemantic activations into sparse, more **monosemantic features**, and how those features can be used to **steer** behavior. Learning objectives: (1) the **superposition** problem — single neurons fire for many unrelated concepts; (2) an SAE learns an overcomplete dictionary of sparse features; (3) many features become human-interpretable ("Golden Gate Bridge," "DNA," a visual texture); (4) **feature steering** — amplifying a feature changes outputs (the "Golden Gate Claude" phenomenon); (5) interpretability enables control and safety. (arxiv)

Need / gap analysis. SAEs and mechanistic interpretability are among the most consequential recent AI-safety advances (Anthropic's *Towards Monosemanticity* and *Scaling Monosemanticity*; DeepMind's Gemma Scope), and Anthropic's "Golden Gate Claude" made feature steering famous. VISxAI's Patchscopes entry (2024) touched LLM self-explanation, but **no explainable lets a general audience browse a feature dictionary and steer a model.** Recent work extends SAEs to VLMs (Pach et al., "Sparse Autoencoders Learn Monosemantic Features in Vision-Language Models," NeurIPS 2025; arXiv:2504.02821), demonstrating that "applying SAE interventions on CLIP's vision encoder directly steers multimodal LLM outputs (e.g., LLaVA), without any modifications to the underlying language model" — offering a striking visual demo. The novelty is a **playable microscope + steering knob**, not a static feature atlas.

Target audience. Students, safety-curious practitioners, policymakers interested in interpretability/control, and a broad public primed by "Golden Gate Claude" coverage.

Interactive visualization design.

- **Superposition demo:** a toy 2D visualization showing why more concepts than neurons forces polysemantic overlap, then how sparsity disentangles them.
- **Feature dictionary browser:** a UMAP/grid of learned features; click one to see its **top activating examples** (text snippets or, for a VLM SAE, top activating images/patches).
- **The steering knob:** a slider that amplifies/suppresses a chosen feature and shows the model's output shifting in real time (precomputed sweep) — e.g., dialing up a "bridge" feature bends generations toward bridges.
- **Monosemanticity meter:** contrast a raw polysemantic neuron (fires for unrelated images/words) vs. a clean SAE feature, visualizing the interpretability gain.

Narrative / pedagogy (scrollytelling arc).

1. *A neuron that means too many things* — polysemanticity/superposition.
2. *Untangling with sparsity* — what an SAE does.
3. *A dictionary of concepts* — feature browser.
4. *Reading a feature* — top activating examples.
5. *Turning the knob* — feature steering ("Golden Gate Claude").
6. *Why this matters* — interpretability → control & safety.

Methodology. Precomputed with an optional live garnish. Use an open pretrained SAE (e.g., Gemma Scope for text, or a CLIP/VLM SAE for the image variant) to extract features, top activating examples, and a **precomputed steering sweep** (outputs at several amplification levels) offline; serialize to JSON. The image (VLM) variant is especially demo-friendly and can pair with a live SmolVLM-256M WebGPU demo. D3 for the feature map/meter; Svelte for the steering slider.

Feasibility (solo). *Medium.* Leverages *existing* open SAEs (no SAE training needed), which greatly reduces effort; main risk is curating compelling, safe steering examples and keeping the feature browser legible. **MVP:** feature dictionary browser + top activating examples + a single precomputed steering knob. **Stretch:** superposition demo, monosemanticity meter, live VLM steering. Estimated effort: 6–8 weeks.

Novelty statement. First VISxAI explainable to let readers *browse and steer* a model's internal features, turning the Anthropic "Golden Gate Claude" result and SAE research into a playable "microscope with knobs" — with an image-model variant (Pach et al., NeurIPS 2025) that has never been presented interactively for a general audience.

Part IV — Evaluated-but-Not-Selected: Tesla Autopilot

The brief asked me to evaluate "How Tesla Autopilot works" (end-to-end nets, occupancy networks, planning). **I recommend against it as one of the five**, for four reasons:

1. **Venue fit:** VISxAI rewards explainables of *AI methods/techniques*; Tesla FSD is a product/systems story. The strongest available narratives (e.g., FSD v12 reportedly replacing ~300k lines of C++ with end-to-end nets, occupancy networks from 2022 AI Day) are corporate/announcement-driven, not method pedagogy, and specifics are partly opaque — community analyses even dispute whether v12 is truly a single end-to-end network. (Futu News)
2. **Model inaccessibility:** There is no open Tesla model to run or introspect; any visualization would rely on secondhand descriptions and reconstructed diagrams, weakening the "interactive, faithful" ideal reviewers prize.
3. **In-browser feasibility:** A credible occupancy-network / BEV demo needs real 3D perception data and heavy Three.js work — high effort, high risk for a solo author, with limited ability to show *true* internals.
4. **Novelty risk:** Autonomous-driving explainers already exist in popular media; a non-authoritative reconstruction is unlikely to win against method-grounded explainables.

If the author is passionate about autonomy, a **stronger reframing** would be a *general* explainable of "end-to-end vs. modular perception + occupancy grids" using an *open* self-driving dataset/model (e.g., nuScenes-based) — but this is still 3D-heavy and riskier than Proposals 3–5. Quantization is the better use of a suggested slot.

Part V — Recommended Prioritization

Build order recommendation:

1. **Proposal 2 (Inference-Time Scaling)** — *build first.* Highest topical relevance, highest solo feasibility (fully precomputed, no in-browser model), and a clean single-control ("thinking budget") hook. Strong candidate for Best Submission.
2. **Proposal 1 (VLM Interpretability)** — *required and highest wow-factor.* Prioritize because it is mandated and visually spectacular; manage the payload/scope risk carefully. The "gaze overlay" is the kind of image that goes viral (as Transformer/Diffusion Explainer did).
3. **Proposal 3 (Quantization)** — *lowest risk, high novelty.* Excellent fallback/parallel build; almost no in-browser ML risk.
4. **Proposal 4 (MoE)** — strong, feasible, novel; slightly more layout-engineering effort.
5. **Proposal 5 (SAEs/Steering)** — highest wow ceiling but depends on curation quality and available open SAEs; build once one of the above ships.

Thresholds that would change this ranking:

- If **live interactivity with the reader's own input** is deemed essential by the author, elevate Proposals 1 and 5 (which pair with live SmolVLM-256M WebGPU) and demote the fully-precomputed ones.
- If **time is the binding constraint** (< 6 weeks), build Proposal 3 (Quantization) first — it is the most self-contained.

- If **a co-author or compute becomes available**, Proposal 1 or 5 becomes more attractive because the offline extraction and 3D/animation budgets can grow.

Submit at least two explainables if bandwidth allows: the required VLM piece plus the Inference-Time Scaling piece maximize both compliance and award odds.

Part VI — Practical Notes for Reviewers' Expectations

- **Lead with the interactive artifact.** The deployed Vercel URL is what reviewers judge; the abstract/paper is secondary and may be ignored. Make the first screen interactive within seconds.
- **Progressive disclosure + multi-level abstraction.** The venue's winners (Transformer Explainer, CNN Explainer) succeed by letting readers start high-level and drill into math on demand. Mirror this.
- **Address misconceptions explicitly.** Each proposal above names a misconception to correct (attention = explanation; more thinking = better; INT4 = broken; experts = subjects; neurons = concepts). Reviewers reward genuine pedagogy.
- **Mobile + performance.** Keep JSON payloads small and lazy-loaded; test WebGPU fallbacks (WebGPU can crash on some Android/mobile GPUs where FP16 is unsupported); ensure the scrollytelling works on a laptop trackpad and phone.
- **Open-source it.** MIT-license the repo (as Polo Club entries do); reviewers and the community value reproducibility, and it strengthens the "stands on its own" requirement.
- **Timeline discipline.** With an August 3, 2026 deadline and a May 5 CFP, a solo author should lock scope by early June and spend July on polish, performance, and copy-editing the narrative.

Suggested filename for download

`VISxAI-2026-Five-Proposals.md`

(Copy the content above into a file with this name to save the deliverable as structured Markdown.)